Technical Report

ECE 26.1 Winter River: A Modular Data Center Design

March 18th, 2026

Leilani Gonzalez — Computer Engineering

Keshav Verma — Electrical Engineering

William McDonald — Electrical Engineering

Ton Dam Lam (Adam) — Electrical Engineering

Ezekiel Mitchell — Computer Engineering

ECE 26.1
Science and Engineering Project Center
College of Science and Engineering Seattle University

TABLE OF CONTENTS

DEFINITIONS 2

I. INTRODUCTION 3

II. SYSTEM ARCHITECTURE 4

A. Proposed Solution 4

B. Power Hierarchy and Node Types 6

III. HARDWARE DESIGN AND INTEGRATION 8

A. Modular Physical Architecture of Nodes 8

B. Baseplate Design 10

B.1 Design Overview 10

B.2 Power Architecture and System Power Budget 10

B.3 PCB Design 12

IV. SOFTWARE DESIGN AND INTEGRATION 17

A. ESP32 Node Firmware 17

B. Event-Driven Communication -- (MQTT) Architecture 18

C. Internal Simulation Engine (Python/SQL) 19

D. Interactive Visualization (Data aggregation) Design Overview 22

V. TESTING AND VALIDATION 23

A. PCB Testing 23

B. Software and Firmware Testing 25

C. Integration Testing 26

VI. CURRENT PROJECT STATUS/ANTICIPATED NEXT STEPS 27

VII. CONCLUSIONS 28

VIII. REFERENCES 28

APPENDIX A. PCB DESIGN AND TESTING 29

DEFINITIONS

Modular – Removable and rearrangeable. Modularity allows the user to assemble the model in different configurations.

Node – The physical electronic element which represents an electrical load within the data center. As a collective, nodes will organize to form a fully functional simulation, representing all main components of a data center. As an example, a node may represent cooling equipment like a chiller.

Baseplate – A surface which separates and distributes power among nodes. The baseplate represents the foundation which a data center is built upon. It consists of two major parts - a laser cut scaffolding structure, and a printed circuit board for power distribution.

USB-C Receptacle – The female part of a USB-C connection, referring to the opening or slot where you insert a USB-C plug. Also referred to as a USB-C port.

USB-C Plug - The male part of a USB-C connection, referring to the metal-tipped end that inserts into a USB-C receptacle. Also referred to as a USB-C connector.

Brownout – Temporary drop in supply voltage below minimum operating threshold, interrupting device operation.

Rack – An enclosure which holds data center equipment. A rack is typically measured by the vertical space it can provide. A common application for an equipment rack is data storage or server space.

MQTT – a lightweight publish/subscribe messaging protocol designed for constrained devices and low-bandwidth, high-latency networks. Used as the primary inter-node communication layer in this system.

LWT – Last Will and Testament – an MQTT feature that allows a client to register a message to be published automatically by the broker if the client disconnects unexpectedly. Used in this system for real-time node disconnect detection.

2N Redundancy – A power redundancy scheme in which every component in the power chain has a completely independent, full-capacity backup. Loss of one entire power path does not interrupt operation.

I. INTRODUCTION

The rapid growth of data center construction and operations has exposed a critical skills gap. Data center employment in the United States grew by 60% from 2016 to 2023, rising from 306,000 to 501,000 workers, yet the supply of qualified personnel continues to fall short of industry demand [1]. In today's environment, hands-on educational solutions that bridge theory and practice are urgently needed.

To address this challenge, AWS seeks a solution to illustrate the primary components of a data center and their associated behaviors. Specifically, AWS requests a modular, tabletop model of a data center that can support the onboarding and training of new data center personnel. The development of this solution is a collaborative effort with a Mechanical Engineering team, combining electrical and mechanical design aspects to deliver a fully functional model.

Winter River is a distributed embedded simulation platform using ESP32 microcontrollers connected through an MQTT network, with a Raspberry Pi 5 acting as the central broker and simulation controller. Each ESP32 node represents a specific data center component: transformers, generators, UPS systems, Power Distribution Units (PDUs), and server racks. Power states propagate through the system using a topological sorting algorithm and stored in a PostgreSQL database, enabling deterministic modeling of cascading failures within the power hierarchy. A physics-based thermal engine models airflow scaling using fan affinity laws and estimates rack temperature rise through convective heat transfer equations, enabling automatic proportional fan control. The system demonstrates realistic 2N power redundancy with generator auto-start logic, UPS battery transitions, and dual-path failover scenarios.

II. SYSTEM ARCHITECTURE

A. Proposed Solution

The proposed solution is a distributed embedded simulation platform using 24 ESP32 microcontrollers connected through an MQTT network with a Raspberry Pi acting as the central broker and simulation controller. Each ESP32 node represents a data center component, such as transformers, generators, UPS systems, PDUs, and server racks.

Power states propagate through the system using a topological sorting algorithm across a directed acyclic graph stored in a PostgreSQL database, enabling deterministic modeling of cascading failures within the power hierarchy. A physics-based thermal engine models airflow scaling using fan affinity laws and estimates rack temperature rise through convective heat transfer equations, enabling automatic proportional fan control. The system demonstrates realistic 2N power redundancy with generator auto-start logic, UPS battery transitions, and dual-path failover scenarios. The resulting platform achieves sub-second cascade propagation across all nodes with proportional fan response within one simulation tick, while supporting hot-swappable modules and real-time visualization through OLED displays and Grafana dashboards.

The architecture comprises the following key elements:

- **Modular Physical Architecture:** A custom PCB baseplate enclosure with integrated power distribution and hot-swappable ESP32+OLED node modules shaped to represent their real-world counterparts.
- **Event-Driven Communication (MQTT):** Publish/subscribe messaging with Last Will and Testament (LWT) for instant disconnect detection, and real-time bidirectional hardware communication.
- **Internal Simulation/Software (Python/SQL):** Power system simulation propagating state changes through a hierarchical DAG relationship, with physics-based thermal and pressure modeling.
- **Interactive Visualization (Data aggregation):**OLED displays on each physical node for direct readout, time-series Grafana dashboards for system-level trends, and a touchscreen display for simulation event triggering (stretch goal).

A visual aid demonstrating how these elements work together can be found in Figure 1 below.

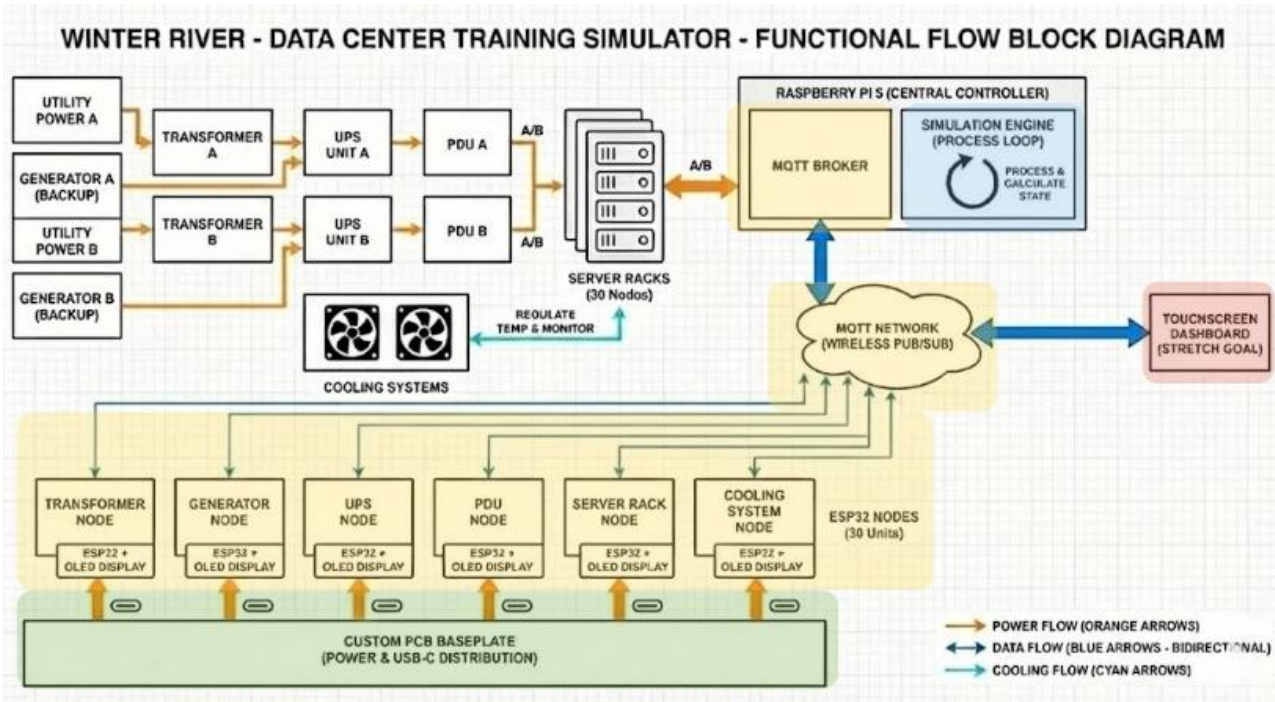


Figure 1: *High-level overview of system architecture*

The top half illustrates simplified power and cooling flows in a real-world data center, which the final model seeks to represent. It shows the power flow from the utility and backup generators down to the server racks, the final load of a data center.

The remainder of the diagram reflects our model's general electronic architecture. Each data center component will be represented by an ESP32 development board (devboard), accompanied by an Organic Light Emitting Diode (OLED) screen which displays its corresponding electrical characteristics. Each ESP32-OLED pair will be secured in a physical enclosure that symbolizes the component function within a data center. For example, the cooling system node enclosure resembles a fan.

All ESP32 nodes communicate via MQTT with a central Raspberry Pi, which runs the simulation logic, aggregates data, and is where data is visualized via a Grafana dashboard. The ESP32s receive power through custom power distribution of PCBs, with a dedicated USB-C plug for each node.

B. Power Hierarchy and Node Types

The system models a complete data center power chain, from utility grid feed through to server rack loads. Each node type was selected in collaboration with the AWS sponsor to represent the most educationally significant components in a real data center. The hierarchy is structured as follows, with power flowing from parent to child nodes:

Node Type	Vin	Vout	Role / Notes

Utility Node	Grid Source	230 kV	Primary grid feed. Duplicated as Utility A and Utility B for 2N redundancy.
MV Switchgear	230 kV	34.5 kV	Medium-voltage distribution switching and protection.
MV/LV Transformer	34.5 kV	480 V	Steps voltage down to facility low-voltage level. Two units (A/B) for redundancy.
Main LV Switchgear	480 V	480 V	Houses the Automatic Transfer Switch (ATS). Monitors Utility vs. Generator feed and switches automatically on fault.
Generator (Backup)	Fuel → Logic	480 V	Modeled with 15-second startup delay. Provides second ATS feed. Activates on loss of both utility paths.
UPS	480 V	480 V	Final line of defense. Battery backup bridges gaps during generator startup. Triggers on redundancy loss.
Rectifier (AC→DC)	480 V AC	48 V DC	Converts AC bus to 48 V DC bus bar that feeds server racks directly.
Server Rack (×24)	48 V DC	—	Primary load. Simulates 12 kW–120 kW per rack depending on type of rack and if configured AI workload.
Cooling System	Power	Airflow	Fan nodes regulate airflow and temperature per zone. Speed is proportional to heat load via fan affinity laws.

Table 1: *Winter River Power Hierarchy. Node Types and Electrical Characteristics*

III. HARDWARE DESIGN AND INTEGRATION

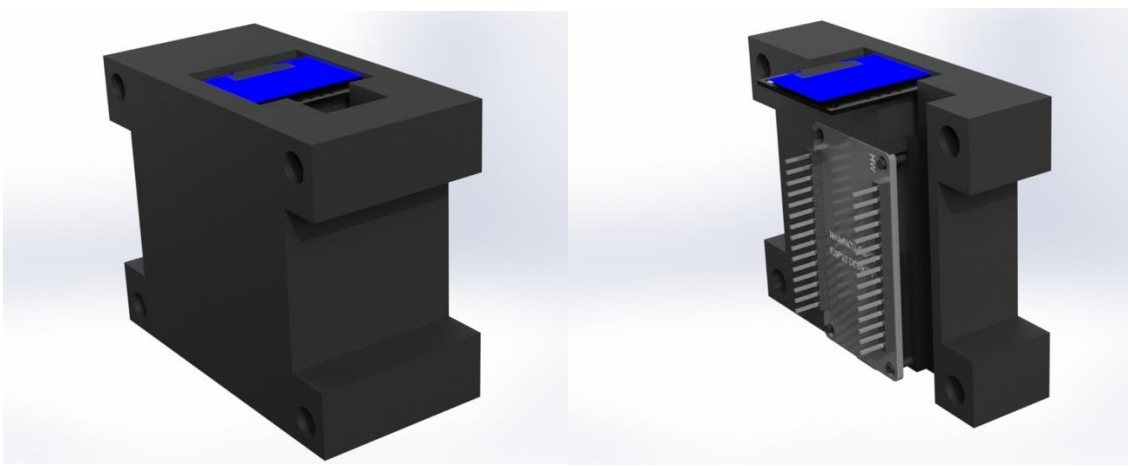
This section provides a detailed description of the hardware aspects of the system previously introduced. It focuses on the design and integration of the ESP32 nodes, their associated OLED displays, the overall electrical architecture, and the custom power distribution PCBs.

A. Modular Physical Architecture of Nodes

Node Design and Components

The goal of each node is to represent an electrical source or load within the data center model. Together, the nodes represent the key electrical and cooling infrastructure found in a typical data center. Each node is comprised of an ESP32 devboard paired with an OLED. The ESP32 was selected for its wireless communication ability, low power draw, and low price. Wireless communication was primarily chosen to simplify the baseplate design. By restricting the baseplate's purpose to power distribution and mounting, the system is easier to design and is less likely of a failure that can hinder system functionality.

Additionally, using a microcontroller allows each node to be individually programmed. This reduces the strain on the central Raspberry Pi for operations such as displaying node information on the OLED. A USB-C type ESP32 devkit was selected over a micro-USB kit to improve the user experience. Unlike micro-USB connectors, USB-C connectors are reversible and are not influenced by plug orientation, simplifying the user experience and supporting modularity by allowing users to easily unplug and plug in nodes as desired.



Figures 2: *Node Design including Enclosure*

Figure 2 illustrates the structural enclosure design of the node. Figure 2 shows the fully assembled node as it appears to the user and the cross-sectional view to convey the ESP32 placement. Key details of this design include structural support for the ESP32 devboard, a chimney-style ventilation channel for airflow, and a wireway for wired I2C communication between the ESP32 and OLED display.

Node Enclosures

The exterior of the node enclosure above does not have details; however, this is a key design element of our project. For the user to identify what the node is, the external enclosure will be labeled and shaped to represent its function. Pictured below is a node enclosure representing a server rack.

Figure 3: *Server Rack Node Enclosure*

The remaining enclosures are still in development. Each node's 3D-printed enclosure is shaped and labeled to cull represent its real-world counterpart.

These node representations were selected in collaboration with our AWS liaison to represent critical data center electrical infrastructure. The project team conducted in-depth research on data center infrastructure, The project team conducted in-depth research during Fall Quarter 2025 to determine which components were most important to include. Table 1 omits details we learned in favor of a user-friendly product and does not include all data center infrastructure; however, it includes the essential components that are most important to demonstrate high-level power flow and facilitate user learning.

B. Baseplate Design

B.1 Design Overview

The baseplate serves as the foundation of our data center model, providing a platform to mount, power, and secure the ESP32 devboards and their associated node enclosures. It integrates both mechanical and electrical aspects:

> **Mechanical:** A structural housing that secures the PCBs, Anker Prime Desktop Charger, dashboard screen, Raspberry Pi, ESP32 devboards, and associated enclosures. Developed in collaboration with the mechanical engineering team.
>
> **Electrical:** A distributed power system used to deliver stable 5V power to 24 ESP32 devboards split across four custom PCBs, each powering 6 devboards.

B.2 Power Architecture and System Power Budget

The Anker Prime Desktop charger provides power to all four PCBs and the Raspberry Pi (RPi). The baseplate power architecture is depicted in Figure 5 below.

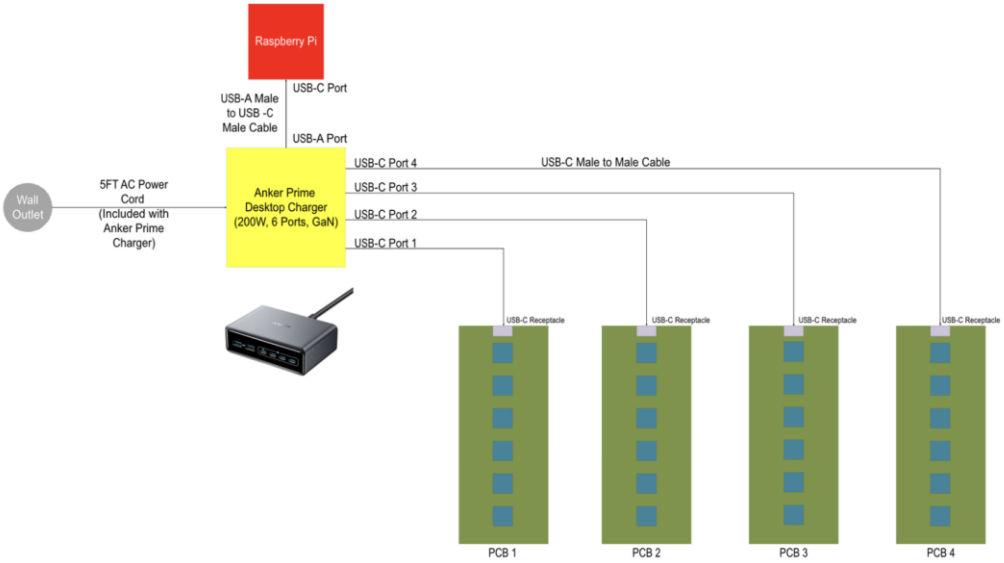


Figure 4: *Baseplate Power Architecture*

The PCBs are solely used for power delivery to the ESP32 devboards, as all communication is handled wirelessly via MQTT. Each PCB receives a stable 5V from an onboard USB-C receptacle connected to a USB port on the Anker Prime Charger via a USB-C male-to-male cable. A wide, thick trace connects the USB-C receptacle to all output USB-C plugs, supplying power to the ESP32 devboards.

Each ESP32 devboard comes pre-assembled with a USB-C receptacle to receive power, which is 5V by standard. The input 5V is stepped down to the required 3.3V using the devboard's low-dropout regulator (LDO) to power the ESP32 chip.

The ESP32's current draw is not constant; rather, drawing current on rapid, high amplitude spikes, especially during MQTT publish/subscribe events and during Wi-Fi transmission. There is no specific current draw range specified by Espressif Systems (the manufacturer of ESP32s), as it is subject to variability depending on the use-case. However, external sources report currents reaching 180-240mA during Wi-Fi transmission, and up to 300-400mA under worst-case operating conditions. Furthermore, each OLED display draws approximately 12mA under standard operating conditions, contributing to this variability.

Accounting for these possibilities, the system was conservatively designed for a worst-case peak current draw of 500mA per ESP32 devboard. Designing around this value provides a balance of efficiency, cost, and complexity. With this margin, there still exists the opportunity to scale the design in the future.

Prior to the PCB design and component selection, the worst-case scenarios were considered. In the event of all ESP32 devboards drawing 500mA simultaneously, and a peak Raspberry Pi current draw of 3A, the following values can be expected:

Parameter	Calculation with Result	
Peak PCB current draw	$500\frac{ma}{devboard} \cdot 6\ devboard\ =\ 3A$	(1)
Peak PCB power consumption	$5V\ \cdot 3A\ =\ 15W$	(2)
Peak Raspberry Pi power consumption	$5V\ \cdot 3A\ =\ 15W$	(3)
Peak system power consumption	$\left(15\frac{W}{PCB} \cdot 6\ PCB\right) + \left(15\frac{W}{RPi} \cdot 1\ RPi\right) = 105W$	(4)

Table 2: *System Power Budget*

In the unlikely event that each ESP32 devboard draws a peak current of 500mA simultaneously, an individual PCB must be capable of 3A as specified in Equation 1. Given that standard USB-C is limited to 3A at 5V, equation 2 calculates a peak power consumption of 15W for an individual PCB. The same principle applies to the Raspberry Pi, which is powered through USB-C, as shown in equation 3.

Likewise, in the improbable event of each PCB and Raspberry Pi drawing 15W, the peak power system consumption equates to 105W as shown in Equation 4. The probability of these scenarios occurring is minuscule; However, designing around these values ensures system stability across all operating conditions and provides generous headroom.

The Anker Prime Charger was selected for its strong specifications and excellent value.

Key specs include:

- **Ports:** 4 USB-C ports, 2 USB-A ports
- **Per port power output:** 100W
- **Max power output:** 200W (cumulatively across all ports)

Given the worst-case cower consumption is 105W, and the Anker Prime Charger is rated for 200W, this charger offers significant headroom, nearly twice the maximum load. Not only does this guarantee smooth operation during simultaneous current spikes, but it also positions the system to be capable of scalability and future expansion.

This charger delivers strong value for the money made possible by Anker's GaN technology. It comes built in with Anker's MultiProtect system, delivering safe power while continuously monitoring the charging temperatures up to 6,000,000 times per day, ensuring connected devices are powered safely [2].

B.3 PCB Design

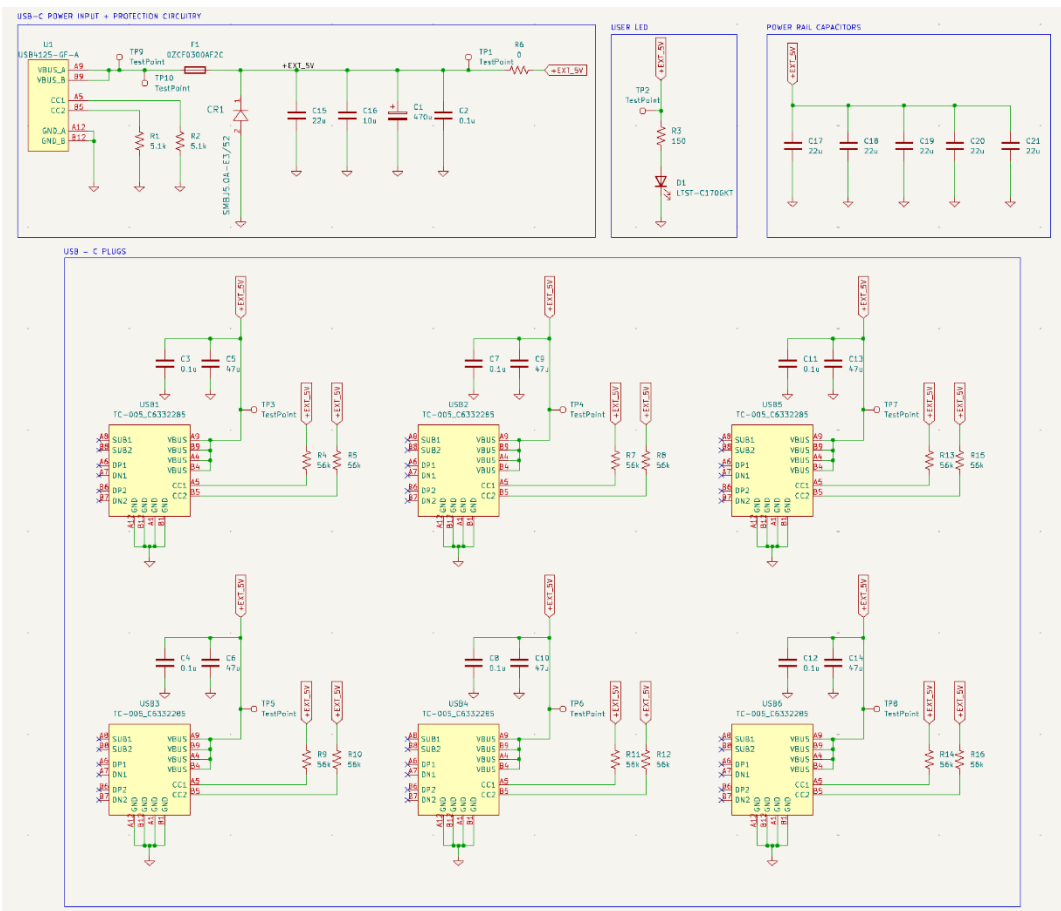


Figure 5: *PCB Schematic*

Protection Circuitry

The PCB design incorporates safety mechanisms that address two common concerns in power input and delivery: overcurrent conditions and electrostatic discharge (ESD) strikes.

1. Overcurrent Protection: In the event of a short circuit or overload, excessive current can damage PCB traces, ESP32 devboards, the Anker Prime Charger, or pose a safety risk. To mitigate this, a positive temperature coefficient (PTC) fuse was selected:
   - **Part #:** 0ZCF0300AF2C
   - **Reference Designator:** F1
   - **Hold Current:** 3A
   - **Trip Current:** 5.2A
   - **Time to trip:** 20 s (max)
   - **Max Voltage:** 15V

This fuse was selected for three main reasons:

- The fuse passes the worst-case calculated current draw of 3A without nuisance tripping, while the trip current rating of 5.2A provides a safe buffer above unexpected transient
- Unlike a one-time fuse, the PTC fuse is self-resetting. As the current approaches the trip level, the fuse temperature heats up, exponentially increasing the resistance and restricting current flow. Once the fault clears, the fuse cools and self-resets
- The fuse can handle up to 100A at 15V without damage or risk of failure. Oftentimes, the fuse can trip in less than 1 second, protecting the ESP32 devboards, the PCB, and the Anker charger

The average time to trip current characteristic curve at 23°C is shown in Figure A1.

2. Electrostatic discharge protection: An electrostatic discharge strike occurs when static charge built on a user's hand or other component suddenly transfers to the PCB, inducing an enormous voltage spike capable of causing severe damage to the PCB traces, components, and ESP32 devboards. The most likely point of contact is cable insertion/removal, where user contact with the USB-C receptacle is possible.

   To mitigate this risk, a Transient Voltage Suppressor (TVS) diode was placed at the input section between the power rail and ground. Placing the TVS diode here allows it to clamp transient voltages at the source before they propagate down the main power bus, protecting the system downstream.

   The TVS diode selected is:
   - **Part #:** SMBJ5.0A-E3/52
   - **Reference Designator:** CR1

- **Working Voltage:** 5V
- **Breakdown Voltage:** 6.4V
- **Clamping Voltage:** 9.2V

Given the power rail has a nominal voltage of 5V, a 5V working voltage ensures no current leakage under normal operation. The 6.4V breakdown voltage provides a 1.4V margin, accounting for occasional transients. When the voltage exceeds 6.4V, the TVS diode clamps the rail to a max voltage of 9.2V, far lower than the voltage of an ESD strike. Each ESP32 devboard's low dropout regulator (LDO) can handle up to a 12V input voltage; All capacitors are rated for 16V, and resistors for 150V, reinforcing the systems' resilience during ESD events.

Power Integrity Considerations

A 470µF bulk electrolytic capacitor was placed at the input, serving as the system's main energy reservoir. It prevents voltage brownout when the Anker charger cannot supply current quick enough.

A 0.1µF ceramic high-frequency bypass capacitor is placed at the power input and locally next to each USB-C plug, to shunt high-frequency noise to ground and ensure a clean power signal.

22µF and 47µF decoupling capacitors bridge the gap between the slow bulk capacitor and the fast high-frequency bypass capacitor. 22µF capacitors are placed throughout the power rail to prevent voltage drop and oppose trace inductance over the 20-inch-long power trace.

47µF capacitors are placed locally at each USB-C plug to serve as a localized energy source for each ESP32 devboard. Their local placement minimizes trace length, reduces parasitic inductance, and enables quicker transient current supply during Wi-Fi transmission spikes. This prevents voltage drop that can result in system instability or crashes.

USB-C Configuration

As shown in Figure A2, plugs that supply power are classified as sources, or a downstream-facing port as per the USB Type-C specification. To establish this role and the plug's current supplying capability, each plug uses 56kΩ ±20% pull-up resistors to the CC pins.

As stated in the USB-C current advertising table referenced in Figure A3, a56kΩ pull-up resistor to 5V supports up to 500mA, aligning with the worst-case peak current of 500mA per ESP32 devboard and the PCB's total 3A limit.

USB Type-C specification classifies receptacles that receive power as sinks or upstream-facing ports. To establish this role, the receptacle uses 5.1kΩ ±20% pull-down resistors to the CC pins in accordance with the requirements outlined in Figure A4.

User LED indicator

A user LED was included on each PCB to indicate power status. The LED draws a minimal amount of current, configured for about 20mA, which is negligible compared to the worst-case 500mA current draw for each node.

Trace Sizing and Current Capacity

The main power trace (VBUS net) was sized in accordance with the IPC 2221 standard to safely handle the worst-case continuous current of 3A with a maximum 10°C temperature rise, given an ambient temperature of 25°C, using 1oz copper on external layers.

Using the Digikey PCB Trace Width calculator, the minimum required width was approximately 53mil, yielding an end-to-end voltage drop of approximately 0.56V over the 20-inch trace at full load. To provide an additional margin against voltage drop, short transients, and short-circuit conditions, the trace was upsized by ~50%, resulting in a width of 80 mil. The calculation results can be found in Figure A6.

Increasing the width from 53mil to 80mil significantly helps combat voltage drop, increasing the worst-case node voltage by approximately 200mV. This provides additional headroom above the devboards' minimum input voltage requirement of 4.07V, further strengthening the power distribution capabilities. Supporting calculations can be found in the appendix with the supporting figure A7.

This wider design has no measurable impact on cost or manufacturability and only requires a small amount of additional board real estate, while significantly improving power integrity and reliability.

Mechanical Considerations + Renders

In collaboration with the mechanical engineering team, the following constraints influenced the design:

- Each ESP32 enclosure measures 80mm x 40mm
- A minimum spacing of 51 mm (2 inches) is maintained between enclosures to ensure the sides of each enclosure are visible to the user and clearly convey what data center component they represent
- At least six M3 mounting holes must be placed throughout the PCB to allow the board to be secured into the mechanical frame of the baseplate

Component courtyards have been drawn on the silkscreen layer of the PCB, outlining the space reserved for the physical enclosures:

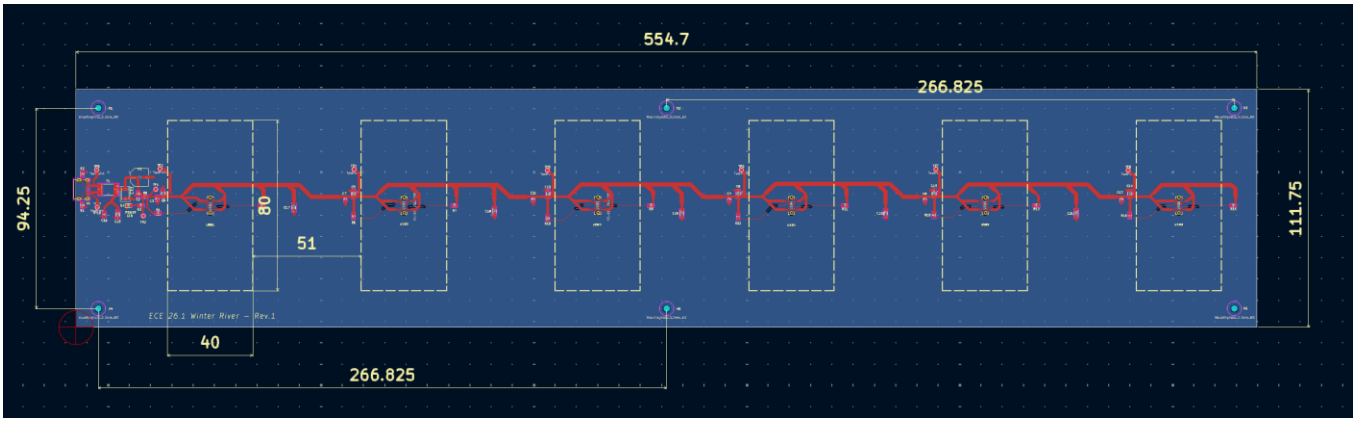


Figure 6: *PCB Layout – Mechanical Considerations*

Six M3 mounting holes were placed throughout, with three placed across the top edge of the board and three placed across the bottom edge. Applying these constraints, the PCB has the following dimensions: 554.7mm x 111.75mm = ~22inches x 4.4 inches. 3D models of the PCB are shown below.

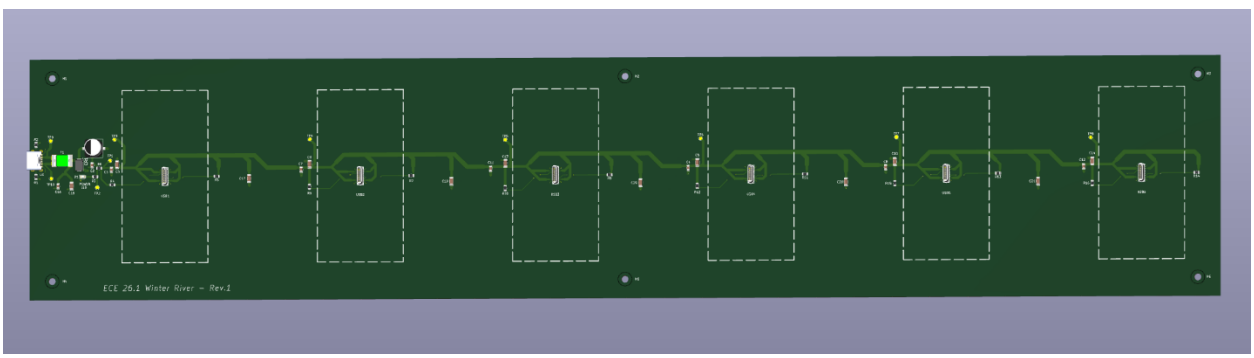


Figure 7a: *PCB 3D Model – Top-Down View*

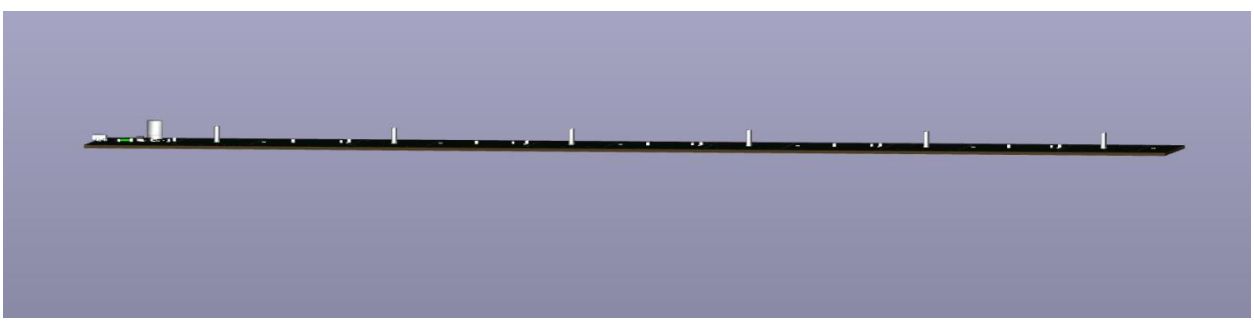


Figure 7b: *PCB 3D Model – Side view*

IV. SOFTWARE DESIGN AND INTEGRATION

The software architecture of Winter River is organized into four interdependent layers: the firmware running on each ESP32 node, the MQTT communication infrastructure, the central simulation engine on the Raspberry Pi, and the visualization layer. Together, these layers create a closed-loop system in which physical events. Such as removing a node from the baseplate, propagating through the digital model, and producing updated readouts on all affected physical nodes within approximately one second.

A. ESP32 Node Firmware

This firmware enables each ESP32 node to function as an independent, networked component in the modular data center model, responsible for communication, local display, and real-time state updates.

Each ESP32 node runs firmware written in C++ using the Arduino framework with the PubSubClient MQTT library. Upon boot, a node performs the following initialization sequence:

1. **Hardware Initialization**

   The ESP32 initializes the OLED display, GPIO interfaces, and internal system parameters required for node operations. Default voltage, status indicators, and identification are loaded into memory.

2. **WiFi Network Connection**

   The node connects to the WiFi access point hosted by the Raspberry Pi controller. The Raspberry Pi serves as the central network for the system, allowing all ESP32 nodes to communicate within a controlled local network without requiring external infrastructure.

3. **MQTT Broker Connection**

   After establishing a connection to the Raspberry Pi's access point, the node connects to the MQTT broker running on the Raspberry Pi. Each node subscribes to topics associated with its specific component and publishes telemetry data describing its current state.

4. **Telemetry & Topic Subscription Setup**

   The node registers its presence on the network and begins publishing telemetry messages. It also subscribes to command topics that allow the simulation engine to broadcast system state updates and operational parameters.

5. **Operational Loop Execution**

   During normal operation, the firmware continuously processes incoming MQTT messages. When new system state data is received, the node updates the OLED display to reflect parameters such as input voltage, operational status, and simulated conditions; providing real-time visual feedback that mirrors the simulated state of the data center.

Template firmware classes have been implemented for all six equipment types: Transformer, Generator, UPS, PDU, Server Rack, and Cooling System. Each template encodes the electrical characteristics appropriate to that component type, including simulated voltage levels, default load parameters, and display formatting specific to its role in the power distribution hierarchy.

This modular firmware structure allows new nodes to be added to the system quickly while maintaining consistent behavior across all components.

B. Event-Driven Communication -- (MQTT) Architecture

MQTT (Message Queuing Telemetry Transport) was selected as the communication protocol for its lightweight footprint, publish/subscribe decoupling, and built-in quality-of-service guarantees. The Mosquitto broker runs on the Raspberry Pi 5 and acts as the central message router for all 24 nodes. We also have decided to use the QoS security level 1 set up for reliable secure MQTT connections.

Each ESP32 node communicates wirelessly with a central Raspberry Pi using MQTT publish/subscribe messaging. The Raspberry Pi hosts the MQTT broker and simulation logic. Nodes publish their operating parameters (load %, voltage, temperature) and subscribe to system-wide updates. The use of MQTT allows scalable node expansion, real-time state updates, and fault detection using the Last Will and Testament (LWT) feature. This architecture ensures that nodes remain independent while still functioning as part of a synchronized system.

Last Will and Testament (LWT) for Disconnect Detection

The LWT mechanism is central to the system's ability to detect physical node removal in real time. When an ESP32 connects to the broker at boot, it pre-registers a LWT message: a JSON payload with present: false and status: REMOVED, stored at its status topic with the retained flag set. For example, if the ESP32 loses Wi-Fi connectivity, because the node was physically unplugged from the baseplate, then the broker automatically publishes this LWT message. The Raspberry Pi's Python MQTT handler receives this message, interprets it as a node removal event, and immediately triggers a full simulation recalculation. The resulting state changes propagate to all downstream nodes within approximately one simulation tick (one second).

This design eliminates the need for polling or heartbeat mechanisms, minimizing network overhead and ensuring that the simulated power hierarchy reflects the physical hardware state with minimal latency. The complete event flow for a node removal is:

1. Physical event: User unplugs a node from the baseplate
2. ESP32 loses Wi-Fi connection and MQTT session
3. MQTT broker publishes LWT payload: {node_id: trans_a, status: offline, present: false}.

4. Python MQTT handler on Raspberry Pi receives the message and marks the node as absent in the PostgreSQL database.
5. Simulation engine recalculates power flow using topological sort
6. Updated states are published via MQTT command topics to all affected downstream nodes
7. Downstream ESP32 nodes receive command messages and update their OLED displays o show offline or INPUT LOSS

For example, unplugging the Transformer A node causes its two downstream UPS nodes, their PDU children, and all associated server rack nodes to transition to INPUT LOSS within one simulation tick (assuming no redundancy). This cascade of state changes spanning three hierarchy levels propagated entirely through the MQTT/Python pipeline.

Quality of Service and Reliability

Status messages are published at QoS Level 1 (at-least-once delivery), ensuring that no state update is silently dropped during brief Wi-Fi congestion. Command messages from the Raspberry Pi to nodes are also sent at QoS Level 1. The retained flag is used selectively for LWT messages, ensuring that a newly connecting node or a restarting Pi receives the last-known state of any currently offline node immediately upon subscription.

C. Internal Simulation Engine (Python/SQL)

The Raspberry Pi runs a Python-based simulation engine (main_simulation.py) that serves as the central orchestrator for the Winter River system. The engine executes on a periodic 1s simulation tick and coordinates four primary subsystems: the Power Flow Logic module, the Thermal Engine, the fan controller, and the MQTT communication handler.

During each simulation tick, the engine receives hardware state updates from ESP32 nodes via MQTT, updates system state within the database, performs power and thermal calculations, and publishes updated values back to the hardware nodes.

A PostgreSQL database stores node metadata, system state variables, event logs, and historical load values. This database architecture enables persistent state tracking, supports time-series visualization through Grafana dashboards, and allows simulation scenarios to be replayed for training and analysis.

C.1 Database Architecture

All system state information is represented within a PostgreSQL database that acts as the single source of truth for the simulation. The primary tables are described below:

Nodes

The Nodes table represents the hierarchical structure of the simulated data center power infrastructure. Each row corresponds to a physical or simulated component such as a utility feed, transformer, UPS, PDU, or server rack.

Fields include:

- node_id – unique identifier for the component
- node_type – equipment type (Transformer, Generator, UPS, PDU, Server Rack, Cooling Node)
- parent_id – reference to the upstream power source
- is_present – boolean indicating whether the node is currently connected
- v_in – simulated input voltage
- v_out – calculated output voltage
- power_kw – current power consumption or load
- status – operational state (ONLINE, OFFLINE, INPUT LOSS)

This structure forms a directed acyclic graph representing the data center power hierarchy, allowing simulation algorithms to propagate electrical state changes through the system.

Thermal Zones

The Thermal Zones table stores environmental state variables associated with cooling regions in the simulated data center. Each row represents a cooling zone containing one or more server racks.

Fields include:

- zone_id – identifier for the cooling zone
- current_temp_f – calculated zone temperature
- airflow_cfm – required airflow rate
- fan_speed_percent – calculated fan speed setting
- thermal_state – zone condition (NORMAL, WARNING, SHUTDOWN)

This table is updated by the Thermal Engine on each simulation tick and provides input for fan control logic and visualization dashboards.

The database serves as the single source of truth for the system. All simulation modules read from and write to the database using SQL queries, ensuring consistency across subsystems and enabling real-time visualization through Grafana's PostgreSQL data source integration.

C.2 Power Flow Logic

The power flow logic module calculates how electrical power propagates through the simulated data center infrastructure. The system models the power distribution network as a hierarchical graph where each node derives its state from its upstream parent.

At the beginning of each simulation tick, the engine performs a traversal of the node hierarchy starting from root nodes representing utility power sources. For each node, the following logic is applied:

1. The node reads the output voltage of its parent node from the database.
2. If the parent node is offline or not present, the node's output voltage is set to zero, indicating loss of input power.
3. If the parent node is active, the node computes its output voltage using a transformation ratio specific to its component type (for example, transformer voltage reduction).
4. The node's calculated state is written back to the database.

This recursive propagation ensures that power failures cascade realistically through the system. If a transformer or utility feed is removed, all downstream components immediately inherit the loss of power.

The logic also supports 2N redundancy, where certain nodes (such as server racks) can receive power from two independent paths. In this case, the rack remains operational as long as at least one upstream power path remains active.

This approach allows the system to demonstrate realistic failover behavior, where disconnecting one power path triggers an automatic transition to the redundant path.

C.3 Thermal Engine

The Thermal Engine was designed by the mechanical engineering team to implement a physics-based model of heat generation and dissipation within the data center model. When executed on each simulation tick, it computes the temperature rise within each cooling zone using simplified thermodynamic relationships.

Heat Generation

Total heat load within a cooling zone is derived from the sum of server rack power consumption values (kW) located within that zone. These values are retrieved from the Nodes table and aggregated to produce the total thermal load.

Airflow

Required airflow is estimated based on the total heat load. Airflow demand scales approximately linearly with heat generation. The required airflow value is then used to compute the fan speed percentage, which is calculated as the ratio between required airflow and maximum fan capacity, constrained to the 0–100% operating range.

Temperature Rise

A simplified convective heat transfer model estimates the inlet temperature rise for server racks as a function of heat load and airflow rate. As airflow decreases relative to thermal load, the predicted rack inlet temperature increases.

If airflow falls below the threshold required to maintain safe operating conditions, the zone transitions into a THERMAL WARNING state. If this condition persists beyond a configurable duration, the system escalates the state to THERMAL SHUTDOWN, simulating protective shutdown behavior common in real data center environments.

After performing these calculations, the Thermal Engine writes updated temperature and airflow values to the Thermal Zones table. These values are then consumed by the fan controller module, which publishes updated PWM commands to the cooling nodes via MQTT. As a result, fan speed changes are reflected on the OLED displays of the cooling modules in real time.

D. Interactive Visualization (Data aggregation) Design Overview

Visualization is key to the user experience. We've aimed for layers of visualization to convey information for different types of users with different understanding levels or intentions of use. The three types of visualization and interaction layers include the OLED display on each node, the time-dependent Grafana dashboard, and the interactive touch screen for simulation event triggering.

The node OLED displays are the simplest electronic visualization in our design. Each node has a built-in 1x2in OLED, which identifies the node and gives very basic electrical or mechanical characteristics specific to that component.

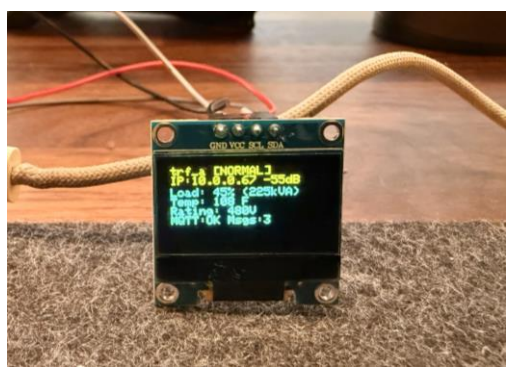


Figure 8: *OLED Display – 480V Transformer*

Figure 8 above exemplifies the OLED display for a transformer node. The top two lines in yellow display the node name "trf_a," the operating condition "[NORMAL]." Below are the important electrical characteristics of the node, such as the load %, temperature, and voltage rating. This information tells the user what the component is, and some basic data on its live function. The electrical characteristics are not fixed; they update the system as it changes. For example, if a server rack is added, it will populate downstream of this transformer, and the load % of "trf_a" will increase.

V. TESTING AND VALIDATION

Validation of the Winter River system is organized across three test categories: PCB hardware testing, software and firmware testing, and integration testing. The fourth category covers educational scenario validation, which will be completed during Spring Quarter.

A. PCB Testing

Objective: Verify that each USB-C plug on the PCBs provides sufficient power to the connected nodes.

Equipment:

- 4x PCBs
- 1x Anker Prime Charger
- 4x USB-C Male to USB-C Male Cable
- 6x ESP32 devboards
- Multimeter
- Test Probes

Procedure:

1. Connect the ESP32 devboards to the PCB USB-C plugs
2. Connect the Anker Prime Charger to the wall outlet and connect the PCBs via USB-C
3. Power on the multimeter and configure it to measure DC voltage
4. Measure voltage at each designated test point (VBUS) for all USB-C plugs
5. Repeat for all PCBs

Results:

Voltages across all plugs and PCBs exceeded the minimum requirement of 4.2V, providing plenty of headroom for the LDO regulators on the ESP32.

Table 2 below illustrates the results:

PCB #	Minimum VBUS (V)	Maximum VBUS (V)
1-4	4.91	4.97

Table 2: *Minimum and Maximum VBUS Readings*

Full measurements and tables are included in tables A4-A7.

Functionality can be visually confirmed as by the power LED indicators on each devboard:

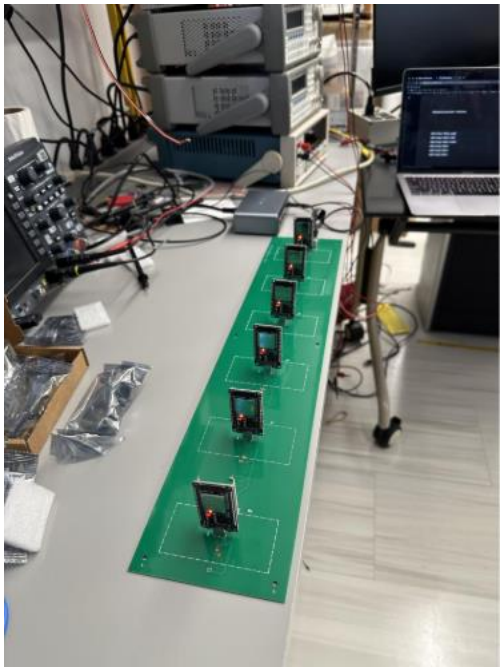


Figure 9: *Individual PCB Test*

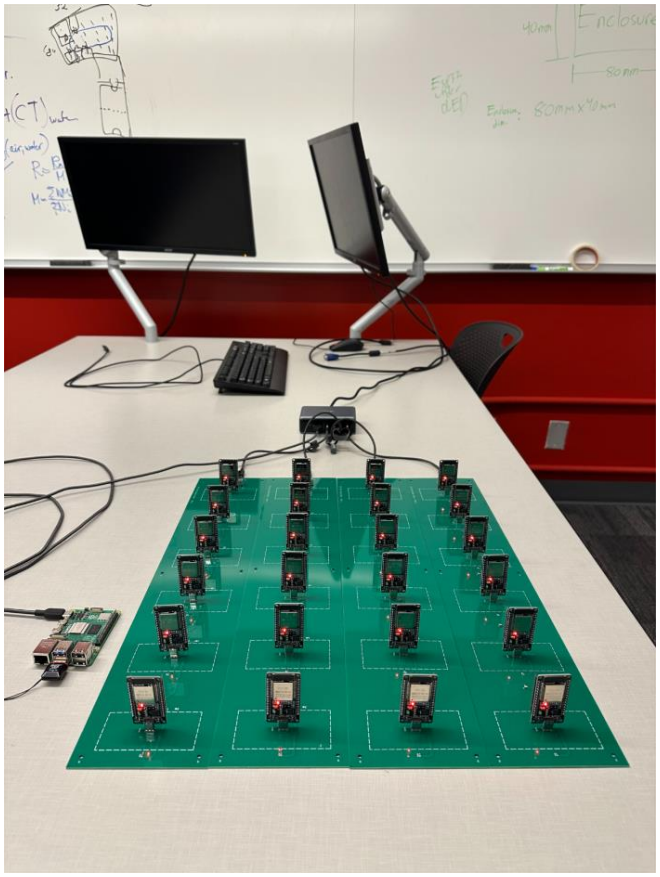


Figure 10: System Test

B. Software and Firmware Testing

Software testing was conducted in two phases: unit-level testing of individual simulation modules on the Raspberry Pi, and firmware verification on isolated ESP32 development boards prior to full system integration.

Unit testing focused on verifying the correctness of the Python simulation modules, including the database interface, power flow calculations, and MQTT messaging logic. Individual functions were tested independently using simulated input values to ensure deterministic system behavior before integrating hardware components.

Firmware testing was performed on individual ESP32 development boards to validate WiFi connectivity, MQTT message handling, and OLED display updates. Each node was flashed with template firmware and connected to the Raspberry Pi access point to confirm successful network connection and topic subscription.

During the Winter quarter development phase, the primary integration testing effort focused on MQTT communication reliability, ensuring that all nodes could reliably publish telemetry data and receive command messages from the central simulation engine.

B.1 MQTT Communication Testing

MQTT communication testing was performed to verify reliable message exchange between ESP32 nodes and the Raspberry Pi MQTT broker. This testing is critical to ensure reliable coordination between distributed nodes and the central simulation engine, which is essential for accurate system behavior.

Each ESP32 node was configured as a client that connects to the Raspberry Pi access point and establishes a session with the Mosquitto MQTT broker running on the Pi. Nodes publish telemetry messages describing their simulated state and subscribe to command topics that provide updated simulation parameters.

Testing focused on three primary communication behaviors:

Node Connection Stability

Multiple ESP32 nodes were connected simultaneously to verify that the MQTT broker could maintain stable client connections. Nodes were repeatedly powered on and off to ensure that reconnection logic functioned correctly, and that devices automatically rejoined the network. Stability was evaluated by observing connection persistence and reconnection time after repeated power cycles.

Message Propagation

Test messages were published from ESP32 nodes to confirm that the Raspberry Pi simulation engine successfully received and processed telemetry updates. The simulation engine then published updated system state messages back to the nodes, verifying bidirectional communication. Successful propagation was confirmed when messages were received and processed within one simulation cycle without data loss.

Node Failure Detection

MQTT's Last Will and Testament (LWT) feature was used to detect node disconnections. When an ESP32 node unexpectedly disconnects, the broker automatically publishes a removal message to the appropriate topic. The simulation engine receives this message and updates the system state to mark the node offline, triggering power state recalculation across the system.

The results demonstrate that the MQTT communication layer provides reliable, scalable, and fault tolerant message exchange between the Raspberry Pi simulation engine and distributed ESP32 nodes.

C. Integration Testing

Integration testing was conducted by running the full software stack on the Raspberry Pi with ESP32 devboards powered by the fabricated PCBs and connected via Wi-Fi. The following system-level behaviors were confirmed:

- Each ESP32 devboard powers on via the USB-C distribution board and runs reliably
- OLED displays correctly show their corresponding node parameters
- MQTT communication with the Raspberry Pi is functional, allowing real-time data aggregation and visualization
- The system remains operational when hot-swapping nodes, free of any disturbance to other nodes

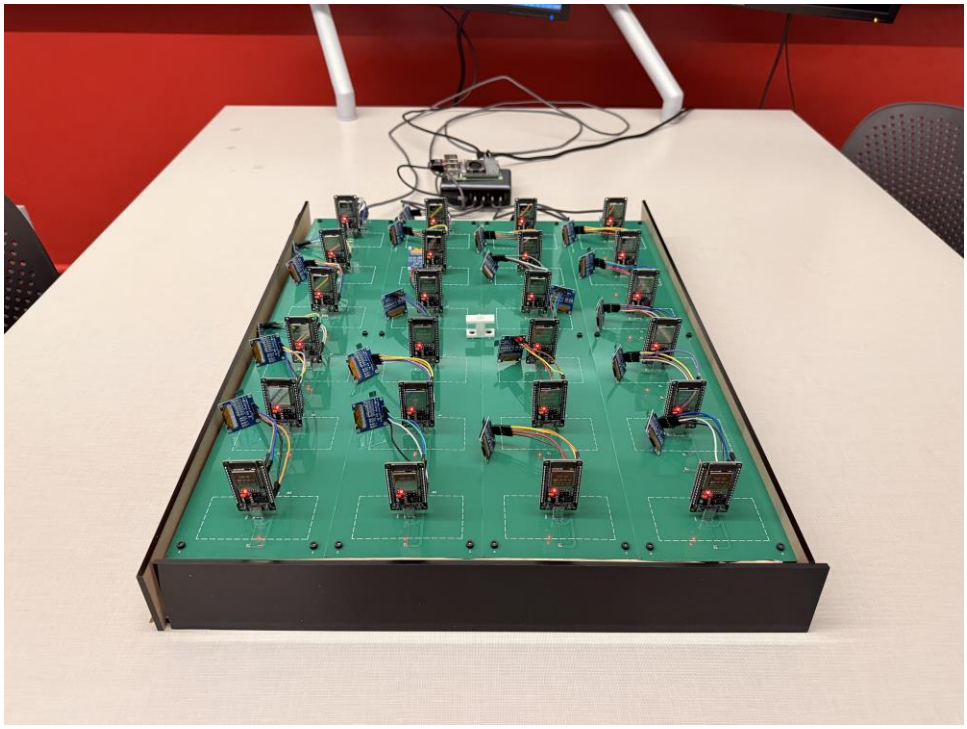


Figure 11: *Current Progress of the Model*

The figure above demonstrates the current progress. There are no enclosures depicted in this figure, but the PCBs are mounted as the first stage of the mechanical baseplate development.

VI. CURRENT PROJECT STATUS/ANTICIPATED NEXT STEPS

Current Status

- All baseline electronics procured and inventoried.
- Four custom PCBs fabricated, assembled, and individually tested — all 24 USB-C outputs verified within specification (4.91V–4.97V VBUS).
- ESP32 devboards powered by PCBs and confirmed operational; OLED displays rendering node parameters correctly via I²C.
- MQTT broker (Mosquitto) runs on Raspberry Pi 5; broker stability confirmed over extended test sessions.

- Firmware templates implemented for all six equipment types (Transformer, Generator, UPS, PDU, Server Rack, Cooling System).
- Hierarchical MQTT communication verified: parent-child power state propagation confirmed across 3-hop test configurations.
- LWT-based disconnect detection validated for single-node removal scenarios.
- Power flow logic simulation validated in software-only mode across all four educational scenarios.
- Grafana dashboard operational: live data from PostgreSQL displayed on gauge, bar, and time-series panels.
- Physical baseplate mechanical structure partially assembled (PCBs mounted; enclosure frame in progress with ME team).
- 3D-printed server rack node enclosure prototyped; remaining enclosure types in development.

Anticipated Next Steps

The following tasks are planned for completion during Spring Quarter 2026:

- Multi-node integration testing with all 24 nodes simultaneously active on the full PCB network.
- Full system 2N redundancy simulation with physical hardware, including generator startup delay and UPS transition.
- Thermal simulation integration: connecting ME-team thermal data to the physics-based Thermal Engine model.
- Completion of all remaining 3D-printed node enclosures.
- Grafana dashboard refinement: additional panels for thermal data, event log, and scenario trigger buttons.
- Touchscreen dashboard UI development (stretch goal) for interactive scenario triggering.
- End-to-end system testing covers all educational failure scenarios with all 30 nodes.
- User manual and technical documentation finalization.
- Final presentation and system delivery to AWS (target: June 2026 Finals Week).

VII. CONCLUSIONS

Winter River is a physical embedded system platform designed to make data center infrastructure training hands-on, safe, and accessible. The system addresses a well-documented skills gap by providing a tabletop-scale, modular representation of real data center power and cooling

infrastructure, where trainees can physically insert, remove, and observe nodes while the system responds with realistic cascading behaviors.

The core technical achievements of the Winter Quarter include the successful design, fabrication, and validation of four custom power distribution PCBs; the implementation of a complete MQTT publish/subscribe architecture with LWT-based disconnect detection; the development of a Python-based simulation engine with power flow propagation and physics-based thermal modeling; and the integration of real-time visualization through OLED displays and a Grafana dashboard.

Hardware testing confirmed that all PCBs deliver stable power within specification across all 24 node positions. Software testing validated the correctness of the power flow algorithm across all four educational failure scenarios in simulation.

The remaining work for Spring Quarter centers on completing the physical assembly with all 30 node enclosures, running full 2N redundancy and thermal failure scenarios on the complete hardware system, and conducting structured educational validation sessions. ECE 26.1 is on track to deliver a fully functional system to AWS by the end of Spring Quarter 2026.

VIII. REFERENCES

[1] Foote, Andrew, and Caelan Wilkie-Rogers. "Employment in Data Centers Increased by More Than 60% From 2016 to 2023 But Growth Was Uneven Across the United States." U.S. Census Bureau, 10 Jan. 2025, www.census.gov/library/stories/2025/01/data-centers.html. (Mar. 2025)

[2] Anker Prime Desktop Charger (200W, 6 ports, Gan) silver a2683141 - best buy, https://www.bestbuy.com/product/anker-prime-desktop-charger-200w-6-ports-gan-silver/JJ858R2R27 (accessed Mar. 17, 2026).

[3] "0ZCF Series Surface Mount PTC Resettable Fuses," Bel Fuse, [Online]. Available: https://www.belfuse.com/media/datasheets/products/circuit-protection/ds-cp-0zcf-series.pdf. Accessed: Mar. 17, 2026.

[4] "TA0357: Overview of USB Type-C and Power Delivery Technologies," STMicroelectronics, [Online]. Available: https://www.st.com/resource/en/technical_article/ta0357-overview-of-usb-typec-and-power-delivery-technologies-stmicroelectronics.pdf. Accessed: Mar. 17, 2026.

[5] Advanced-monolithic. (n.d.-a). http://www.advanced-monolithic.com/pdf/ds1117.pdf

APPENDIX A. PCB DESIGN AND TESTING

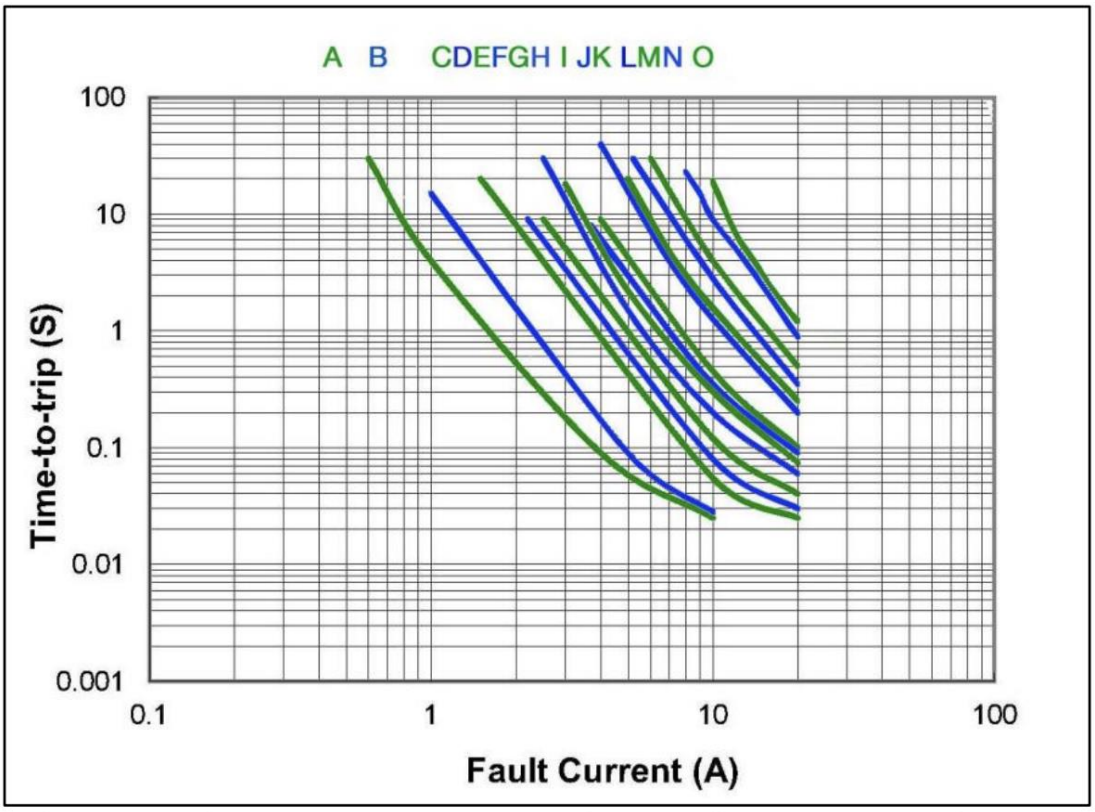


Figure A1: Average Time Current Characteristics Curve at 23°C [3]

As indicated the datasheet, the fuse falls under category "L."

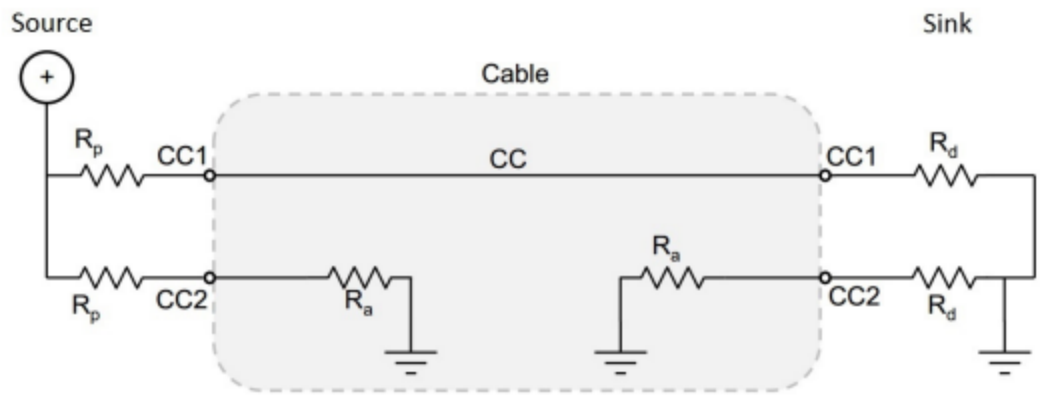


Figure A2: CC logic pullup and pulldown termination [4]

The following table provides the values to be used for Rp or current source.

Table 2. Source CC termination (Rp) requirements

Source Current Capability	Current Source to 1.7V - 5.5V	Rp pull-up to 3.3V ±5%	Rp pull-up to 4.75V - 5.5V
Default USB power	80µA ±20%	36kΩ ±20%	56kΩ ±20%
1.5A @5V	180µA ±8%	12kΩ ±5%	22kΩ ±5%
3.0A @5V	330µA ±8%	4.7kΩ ±5%	10kΩ ±5%

Figure A3: CC Pull-up resistor (Rp) requirements [4]

The following table provides the values to be used for Rd or Sink CC termination.

Table 3. Sink CC termination (Rd) requirements

Rd setting	Nominal Value	Max Voltage on pin	Power Capability detection
±20% voltage clamp	1.1V	1.32V	No
±20% resistor to GND	5.1kΩ	2.18V	No
±10% resistor to GND	5.1kΩ	2.04V	Yes

Figure A4: CC Pull-down resistor (Rd) requirements [4]

CURRENT (I)

3	A

THICKNESS (T)

1	oz/ft²

TEMPERATURE RISE (T_{RISE})

10	°C

AMBIENT TEMPERATURE

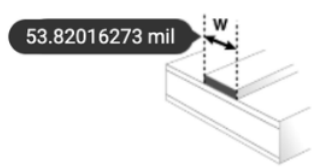
25	°C

TRACE LENGTH

20	in

Figure A5: DigiKey Trace Width Calculator Input Parameters

Minimum Trace Width



External Layers in Air

REQUIRED TRACE WIDTH (W)

53.82016273	mil

RESISTANCE

0.1875345019	Ω

VOLTAGE DROP

0.5626035056	V

POWER LOSS

1.687810517	W

Figure A6: Digikey Trace Width Calculation Results

Table A1: AMS1117-3.3 LDO Typical Performance [5]

Parameter	Conditions	Typical	Maximum
Dropout Voltage (Vin-Vout)	Iout = 800mA	1.1V	1.3V

As provided in the AMS1117 datasheet, a current draw of 800mA has a worst-case dropout voltage of 1.3V. Therfore, the minimum Vin can be calculated as:

$$\boldsymbol{Vin_{\min(800mA)} = 3.3V + 1.3V = 4.6\,V} \qquad \text{Eq. A1}$$

While the LDO's on our devboards are capable of drawing higher currents, the ESP32 chip is not expected to draw more than 500mA, the worst-case current draw scenario we previously established.

Since the dropout voltage of the AMS 1117-3.3 scales linearly with load current the worst-case dropout voltage for this design can be calculated:

$$\boldsymbol{V_{dropout(500mA)} = 1.3V \cdot \frac{500mA}{800mA} = 0.81\,V} \qquad \text{Eq. A2}$$

To account for temperature effects, power supply dips, and other non-idealities, we add a margin to the scaled dropout voltage:

$$\boldsymbol{V_{dropout(500mA)} = 0.8V + 0.1V \approx 0.9\,V} \qquad \text{Eq. A3}$$

Therefore, minimum input voltage for each ESP32 devboard can be calculated:

$$\boldsymbol{Vin_{min} = 3.3V + 0.9V = 4.2\,V} \qquad \text{Eq. A4}$$

Therefore, the main power trace must be designed such that the voltage at each devboard never drops below 4.2V.

PCB Trace Voltage Drop Calculations

Note: The voltage drops calculated below only apply for 3A current draw on power rail, which is very unlikely. This is a worst-case scenario calculation.

Given that the thickness of copper is 35µm, and the resistivity of copper is $1.68 \cdot 10^{-8}\ \Omega \cdot m$, the voltage drops for both trace lengths can be computed as shown below.

Cross-Sectional Area: A = $width \cdot thickness$

Resistance: $R = \rho \cdot \frac{L}{A}$

$$R = \rho \cdot \frac{L}{A}$$

Trace 1 – 80 mil

Length: L = 20 in = 0.508 m
Width: 80 mil = 0.002032 m

Table A2: Voltage drop calculation with an 80-mil trace

Parameter	Calculation with Result	
Area	$A = (0.002032m)(35\mu m) = 7.11 \cdot 10^{-8}\, m^2$	Eq. A5
Resistance	$R = (1.68 \cdot 10^{-8}\ \Omega \cdot m)\left(\frac{0.508\, m}{7.11 \cdot 10^{-8}\, m^2}\right) = 0.120\ \Omega$	Eq. A6
Voltage drop at peak current	$V_{drop} = (3A \cdot 0.120\Omega) = 0.36\, V$	Eq. A7

Trace 2 – 53 mil

Length: L = 20 in = 0.508 m

Width: 53 mil = 0.0013462 m

Table A3: Voltage drop calculation with a 53-mil trace

Parameter	Calculation with Result	
Area	$A = (0.0013462m)(35\mu m) = 4.71 \cdot 10^{-8}\, m^2$	Eq. A8
Resistance	$R = (1.68 \cdot 10^{-8}\, \Omega \cdot m)\left(\frac{0.508\, m}{4.71 \cdot 10^{-8}\, m^2}\right) = 0.181\, \Omega$	Eq. A9
Voltage drop at 3A	$V_{drop} = (3A \cdot 0.181\Omega) = 0.54\, V$	Eq. A10

Finally, the worst-case voltage the last ESP32 node can be calculated as follows:

Trace Width	Voltage Drop	
80-mil	$5V - 0.36V = 4.64\, V$	Eq. A11
53-mil	$5V - 0.54V = 4.46\, V$	Eq. A12

By increasing the main power trace width to 80 mil, the voltage drop is reduced and provides an approximate 200mV margin. This reinforces the design's capabilities and ensures that, even under worst-case scenario, all ESP32 devboards remain operational.

PCB Test Results

Table A4: Voltage Readings for PCB 1

PCB #1	VBUS	Acceptable? (Y/N)
USB-C Plug 1	4.95 V	Y
USB-C Plug 2	4.94 V	Y
USB-C Plug 3	4.94 V	Y
USB-C Plug 4	4.93 V	Y
USB-C Plug 5	4.92 V	Y
USB-C Plug 6	4.91 V	Y

Table A5: Voltage Readings for PCB 2

PCB #2	Voltage Reading (VBUS)	Acceptable? (Y/N)
USB-C Plug 1	4.97 V	Y

USB-C Plug 2	4.94 V	Y
USB-C Plug 3	4.94 V	Y
USB-C Plug 4	4.94 V	Y
USB-C Plug 5	4.93 V	Y
USB-C Plug 6	4.91 V	Y

Table A6: Voltage Readings for PCB 3

PCB #3	**Voltage Reading (VBUS)**	**Acceptable? (Y/N)**
USB-C Plug 1	4.95 V	Y
USB-C Plug 2	4.94 V	Y
USB-C Plug 3	4.93 V	Y
USB-C Plug 4	4.94 V	Y
USB-C Plug 5	4.91 V	Y
USB-C Plug 6	4.91 V	Y

Table A7: Voltage Readings for PCB 4

PCB #4	**Voltage Reading (VBUS)**	**Acceptable? (Y/N)**
USB-C Plug 1	4.96 V	Y
USB-C Plug 2	4.94 V	Y
USB-C Plug 3	4.94 V	Y
USB-C Plug 4	4.94 V	Y
USB-C Plug 5	4.92 V	Y
USB-C Plug 6	4.91 V	Y

Table A8: PCB Bill of Materials (BOM)

COMMENT	**DESIGNATOR**	**FOOTPRINT**	**JLCPCB Part #**
TC-005_C6332285	USB1-6	Connector_USB:USB-C-SMD_TC-005	C6332285
USB4125-GF-A	U1	Connector_USB:USB_C_RECEPTACLE_4125_GCT	C3151650
TestPoint	TP1-10	TestPoint:TestPoint_Pad_D1.5mm	
0	R6	Resistor_SMD:R_0805_2012Metric	C17477
56k	R4, R5, R7-16	Resistor_SMD:R_0805_2012Metric	C17756
150	R3	Resistor_SMD:R_0805_2012Metric	C17471

5.1k	R1, R2	Resistor_SMD:R_0805_2012Metric	C27834
0ZCF0300AF2C	F1	Fuse:FUSE_BEL_FUSE_0ZCF0075AF2C	C3762468
LTST-C170GKT	D1	LED_SMD:LED_0805_2012Metric	C125090
SMBJ5.0A-E3/52	CR1	SMB (DO-214AA):SMBJ5.0A	C61069
10u	C16	Capacitor_SMD:C_0805_2012Metric	C1713
22u	C15, C17-21	Capacitor_SMD:C_1206_3216Metric	C77088
47u	C5, C6, C9, C10, C13, C14	Capacitor_SMD:C_1206_3216Metric	C385047
0.1u	C2-4, C7, C8, C11, C12	Capacitor_SMD:C_0805_2012Metric	C49678
470u	C1	Capacitor_SMD:CAP-SMD_ EEEFK1C471P	C79121